

DATA ANALYTICS PROJECT

Cloud Computing · PySpark Project

Advanced Spark Commands with Python

Ali Vega

Curso: Cloud Computing

Mayo 2026

Python • PySpark • Spark DataFrames • Jupyter Notebook

Implementación de comandos avanzados de Spark utilizando PySpark para análisis distribuido, particionado y procesamiento de datos.

Objetivo

Aplicar comandos avanzados de Apache Spark con PySpark para procesamiento distribuido y análisis de datos.

- Configurar SparkSession con Python
- Importar y procesar dataset Housing con PySpark
 - Aplicar consultas, agrupamientos y agregaciones
- Implementar particionado y exportación de datos

Tecnologías

Stack utilizado en el proyecto



```
from pyspark.sql import SparkSession
from pyspark.sql.functions import count, avg
```

```
import pandas as pd
```

```
archivo_exportado = pd.read_csv("output_housing_python.csv")
archivo_exportado.head()
```

	id	date	price	bedrooms	bathrooms	sqft
0	5466700450	20141015T000000	250000.0	4	1.75	1500
1	8856004327	20140509T000000	248000.0	4	3.00	1500
2	1005000062	20140801T000000	299000.0	2	1.00	1500
3	705700240	20150218T000000	380000.0	4	2.50	1500
4	7349650330	20140609T000000	270000.0	4	2.75	1500

5 rows × 7 columns

Procesamiento

Importación, exploración y preparación del dataset Housing en PySpark.

- Lectura del archivo CSV con `spark.read.csv()`
- Configuración de `header=True` e `inferSchema=True`
- Validación de estructura con `printSchema()`
- Visualización inicial con `show()`

```
housing = spark.read.csv(  
    r"C:\Users\alive\Downloads\kc_house_data (1).csv",  
    header=True,  
    inferSchema=True  
)
```

```
housing.printSchema()
```

```
root  
|-- id: long (nullable = true)  
|-- date: string (nullable = true)  
|-- price: double (nullable = true)  
|-- bedrooms: integer (nullable = true)  
|-- bathrooms: double (nullable = true)  
|-- sqft_living: integer (nullable = true)  
|-- sqft_lot: integer (nullable = true)  
|-- floors: double (nullable = true)  
|-- waterfront: integer (nullable = true)
```

▶ `housing.show(5)`

```
... +-----+-----+-----+-----+-----+-----+-----+-----+  
|      id|      date|  price|bedrooms|bathrooms|sqft_living|sqft_lot|floors|waterfront|  
+-----+-----+-----+-----+-----+-----+-----+-----+  
|7129300520|20141013T000000|221900.0|3|1.0|1180|5650|1.0|  
|6414100192|20141209T000000|538000.0|3|2.25|2570|7242|2.0|  
|5631500400|20150225T000000|180000.0|2|1.0|770|10000|1.0|  
|3127300075|20141100T000000|601000.0|4|2.0|1060|5000|1.0|
```

Análisis PySpark

Consultas distribuidas y métricas analíticas sobre el dataset Housing.

- Agrupamiento y ordenamiento por zipcode
- Conteo de viviendas por código postal
- Cálculo de precio promedio y tamaño promedio
- Análisis por habitaciones, baños y precio

```
housing.groupBy("zipcode") \
    .agg(count("*").alias("total_casas")) \
    .orderBy("total_casas", ascending=False) \
    .show(20)
```

```
... +-----+-----+
|zipcode|total_casas|
+-----+-----+
| 98103|         602|
| 98038|         590|
| 98115|         583|
```

```
housing.groupBy("zipcode") \
    .agg(
        avg("price").alias("promedio_precio"),
        avg("sqft_living").alias("promedio_tamano")
    ) \
    .orderBy("zipcode") \
    .show(20)
```

```
... +-----+-----+-----+
|zipcode|promedio_precio|promedio_tamano|
+-----+-----+-----+
| 98001|280804.6906077348|1900.8563535911603|
| 98002|234284.0351758794|1627.7437185929648|
| 98003|294111.27857142856|1928.8821428571428|
```

Optimización / repartition

Particionado y distribución de datos en PySpark.

- Creación de particiones con `repartition(4)`
- Redistribución de datos para procesamiento distribuido
- Validación con `getNumPartitions()`
- Optimización del rendimiento en Spark

Se utiliza `repartition()` para dividir el DataFrame en 4 particiones. Esto permite simular el procesamiento distribuido de datos dentro de Spark.

```
housing_partition = housing.repartition(4)
```

PLACEHOLDER GRANDE
SCREENSHOT / OUTPUT / NOTEBOOK

Se verifica el número de particiones del DataFrame utilizando `getNumPartitions()`.

```
housing_partition.rdd.getNumPartitions()
```

... 4

Insights

Hallazgos principales obtenidos mediante análisis distribuido con PySpark.

- Determinados zipcodes concentran mayor número de viviendas
- Existen diferencias relevantes en precio promedio entre códigos postales
 - El tamaño de las propiedades influye en el valor promedio de venta
- PySpark permitió procesar métricas analíticas utilizando operaciones distribuidas

Conclusión

Cierre ejecutivo del proyecto

- PySpark permitió procesar y analizar el dataset Housing mediante operaciones distribuidas
- `groupBy()`, agregaciones y `repartition()` facilitaron consultas y optimización en Spark
- La práctica fortaleció habilidades aplicadas en análisis distribuido y procesamiento con PySpark